

Supplementary Material for An Empirical Evaluation of Message Delivery Reliability and Recovery Characteristics in Redis Streams and NATS JetStream

Victor Bona

September 2026

This supplement retains full analytical arguments, the original campaign’s detailed diagnostics, and the follow-up’s complete condition summaries. Historical numerical observations are unchanged. Statements in the original results concern Redis 7.4.2 and NATS Server 2.10.24 in the original accumulated-storage campaign. They are not descriptions of the subsequent fresh-store deployments, reported separately below and in the main article.

1 System model and formal results

1.1 Events, failure scope, and assumptions

An application assigns each message a unique identifier. We distinguish publication invocation p , publication confirmation q , broker delivery d , client receipt $\hat{d}$, consumer failure f , redelivery receipt $\hat{r}$, and acknowledgment completion a . All request and recovery event intervals use a common Linux monotonic clock on the driver node. Broker telemetry uses the broker node’s clock and is never subtracted from driver timestamps. The broker’s internal delivery timestamp d is not directly observed; replacing it with $\hat{d}$ without qualification would misstate the timer origin.

The fault model is a fail-stop consumer process killed with **SIGKILL** while its message is unacknowledged. In the follow-up’s matched live controls, f instead denotes the scheduled reference event while the victim remains alive without acknowledging; the same residual algebra applies without requiring a death. The broker, network, retained payload, group/consumer state, and surviving consumer remain available. There is no message expiry, trimming, competing claim, negative acknowledgment, acknowledgment extension, or backoff change during the recovery episode. The timeout threshold is $\Delta > 0$ and failure occurs at age $h = f - d$, with $0 \leq h < \Delta$. These are assumptions of the isolated formal model; they do not characterize every production execution. The controller sets age relative to client receipt, so exact broker age is unobserved. Idealized eligibility also abstracts from timer quantization and assumes no relevant broker-clock discontinuity.

For a bounded redelivery theorem, additional timing assumptions are required. Let P bound the time from eligibility until a reclamer or expiry mechanism selects the entry, and let B bound subsequent scheduling, demand availability, transport, and client observation. Redis command execution time and scheduler delay must be included in P ; its configured sleep alone is not such a bound. JetStream also requires pull demand, so B cannot silently be set to zero.

Theorem 1.1 (Residual timeout and conditional recovery bound). *Under the preceding assumptions, let selection be forbidden before $d + \Delta$, occur at some $s \in [d + \Delta, d + \Delta + P]$, and be observed after an additional delay $b \in [0, B]$. Then*

$$\Delta - h \leq \hat{r} - f \leq \Delta - h + P + B.$$

If the corresponding upper-bound premise is removed and admissible selection or observation delays are unbounded, no finite uniform upper bound on redelivery latency follows.

Proof. By definition, $\hat{r} = s + b$ and $f = d + h$. Hence

$$\hat{r} - f = \Delta - h + (s - (d + \Delta)) + b.$$

The two final terms lie in $[0, P]$ and $[0, B]$, respectively, proving both inequalities. If either delay is unbounded, for any proposed finite upper bound choose an admissible delay exceeding it. The corresponding execution violates that proposed bound without violating eligibility or retention. $\square$

Corollary 1.2 (Why client-observed timer excess can be negative). *Write $\hat{d} = d + u$, where $u \geq 0$ is the initial delivery-to-receipt delay, and let $w = s - (d + \Delta)$. Then*

$$(\hat{r} - \hat{d}) - \Delta = w + b - u.$$

Thus a measured receipt-to-redelivery interval slightly below Δ does not alone refute server-side timeout eligibility.

Proof. Substitute $\hat{r} = d + \Delta + w + b$ and $\hat{d} = d + u$, and cancel d . $\square$

Proposition 1.3 (No autonomous Redis reassignment from new-message reads). *In the stated Redis new-message-read-only model, there exists an infinite execution with an available broker and surviving consumer, a retained message pending for a failed consumer, and no redelivery, if the survivor performs only new-message reads and no reclamation or pending-history recovery.*

Proof. Deliver the only message to the consumer that subsequently fails, thereby placing it in that consumer's PEL history. Repeat `XREADGROUP` with `>` and no `CLAIM` option from the survivor. Each read considers previously undelivered entries, while the retained entry remains pending for the failed consumer. With no further publication or state-changing recovery operation, every repetition preserves this state. The infinite repetition witnesses the claim. $\square$

The proposition is a liveness counterexample, not a defect claim. A Redis application supplies the missing recovery operation. Likewise, a JetStream pull consumer without future pull demand has no unconditional delivery-time guarantee. The experimental contrast concerns who initiates eligibility/reassignment and the timing of the configured client policy.

1.2 Concurrency and finite-window throughput

Definition 1.4. For $N > 0$ completed messages, an integer $C \geq 1$ of consumers, and a drain interval of duration $T > 0$, let $X = N/T$ be acknowledged throughput. Message i occupies one consumer during a nonoverlapping-on-that-consumer interval $[r_i, a_i]$, from its successful receive-request invocation to acknowledgment completion. Write $R_i = a_i - r_i$ and $\bar{R} = N^{-1} \sum_i R_i$.

Theorem 1.5 (Consumer occupancy bound). *If every interval lies within the measured drain window and each of C consumers has at most one such interval active at a time, then*

$$X\bar{R} \leq C.$$

More precisely, with $U = (CT)^{-1} \sum_i R_i \in [0, 1]$, one has $X = CU/\bar{R}$ whenever $N > 0$ and $\bar{R} > 0$.

Proof. For consumer j , its intervals have disjoint interiors and are contained in a window of length T , so their lengths sum to at most T . Summing over all C consumers gives $\sum_i R_i \leq CT$. Since $X\bar{R} = (N/T)(\sum_i R_i/N) = \sum_i R_i/T$, the inequality follows. Substituting the definition of U yields the identity. $\square$

For two individual trials with positive mean cycle times, this gives the exact explanatory decomposition:

$$\frac{X(C_2)}{X(C_1)} = \frac{C_2 U(C_2) \bar{R}(C_1)}{C_1 U(C_1) \bar{R}(C_2)}.$$

The identity does not generally survive separately averaging throughput, occupancy, and cycle time across trials. Reported ratios of condition-mean throughputs are distinct summaries. Approximately stable cycle cost and occupancy are sufficient for approximately linear scaling within this model. More generally, linear scaling requires $U(C)/\bar{R}(C)$ to remain approximately constant; increasing C alone does not prove it. For example, admissible conditions with $U = 1$, $C_1 = 1$, $\bar{R}(C_1) = 1$, $C_2 = 2$, and $\bar{R}(C_2) = 3$ give throughputs 1 and $2/3$. This counterexample rules out a universal monotonicity claim under unrestricted contention. Our analysis checks the occupancy inequality directly against every retained concurrency-trial event interval.

1.3 Acknowledgment, storage, and application effects

Proposition 1.6 (Conditional stable-storage implication). *Assume (i) a successful synchronization makes the message and all metadata required for its recovery stable, (ii) subsequent failures preserve that stable state, (iii) recovery interprets it correctly, and (iv) confirmation occurs only after that synchronization completes successfully. Then every confirmed message is recoverable after such a failure. If confirmation is instead permitted before stabilization, confirmation alone does not imply recoverability.*

Proof. Under (iv), every confirmed message has already undergone the successful synchronization of (i). Its recovery state survives by (ii), and (iii) recovers it. For the second claim, choose an allowed execution in which confirmation precedes synchronization and a failure occurs strictly between them. Let the failure discard all unstable state. The message was confirmed but is not recoverable, providing a counterexample. $\square$

This is a theorem about a storage contract, not a formal verification of either product. In particular, the historical NATS 2.10.24 append path invokes synchronization without checking its returned error in that path, whereas Redis 7.4.2 treats an always-mode synchronization error as fatal [7, 3]. NATS 2.15.0 checks and propagates the corresponding append synchronization error [8]. The historical error-handling statement must not be generalized to that current release. The experiment injects no I/O faults and does not establish assumptions (i)–(iv) for every implementation execution. The source distinction prevents us from treating a mode name as an unconditional proof of identical fault handling.

Consider an interval containing $F \geq 1$ complete, nonoverlapping synchronization operations. If each occupies at least $f_{\min} > 0$ time, every counted confirmation is assigned to one completed operation, and at most an integer $b \geq 1$ of confirmations are assigned to each operation, those operations can confirm at most bF messages while occupying at least $Ff_{\min}$ time. Their service rate is therefore at most $b/f_{\min}$. This bound applies to the explicitly serialized synchronization stage; overlapping stages and the rest of the pipeline require a different model. It explains why group commit and outstanding-publication count can affect cost without asserting a fixed number of hardware synchronization calls per message.

Proposition 1.7 (The application-effect/acknowledgment gap). *For a non-idempotent external effect and a broker acknowledgment that are separate operations, changing only their order cannot guarantee exactly one completed effect under arbitrary fail-stop interruption and eventual redelivery of unacknowledged work.*

Proof. If the effect precedes acknowledgment, interrupt after the effect and before acknowledgment. The retained unacknowledged message may be redelivered and its effect repeated. If acknowledgment precedes the effect, interrupt after acknowledgment and before the effect; the broker has no remaining obligation to redeliver, so the effect can be absent. Either order admits a counterexample. An atomic transaction, durable application deduplication, or an idempotent effect adds an assumption not present in this model. $\square$

1.4 What zero observed failures establishes

For $n \geq 1$ independent identically distributed Bernoulli trials with failure probability p , observing zero failures has probability $(1 - p)^n$. For $0 < \alpha < 1$, setting this to α gives the one-sided exact upper confidence limit

$$p_{\text{upper}} = 1 - \alpha^{1/n}.$$

At $n = 10$ and $\alpha = 0.05$, this is approximately 0.259, not zero [1]. Our heterogeneous configurations and temporally adjacent messages are not automatically independent Bernoulli draws from one population. We therefore report exact observed counts and do not turn millions of message completions into a misleading claim of extremely small universal failure probability.

1.5 Machine-checked subset and proof boundary

Ten declarations in the artifact’s Lean 4.24.0 model check integer-time residual and receipt-offset identities, a division-free consumer-capacity lemma, and finite effect/acknowledgment crash witnesses [2, 5]. The retained homelab compiler receipts identify the source, compiler, exit status, and transitive logical dependencies. No admitted proof, custom axiom, or native-evaluation shortcut is used. The capacity lemma assumes each consumer’s busy-time bound; the interval argument deriving that premise remains the hand proof above. This checks selected abstract statements, not broker implementations, measurement software, storage survival, or statistical coverage.

2 Original statistical method and attempt history

The randomization seed is 20260923. The experimental unit is a trial, with pairing by block for treatment ratios. For each condition we report the arithmetic mean of its completed planned

trials and a percentile bootstrap 95% confidence interval using 10000 resamples of those trial indices. Conditions have ten completed trials except Redis-always publication, which has nine completions and one failed planned trial. That failure is reported separately, not assigned an artificial throughput or latency. A paired ratio is the ratio of resampled condition means under the same resampled block indices, not the mean of arbitrary per-message ratios. Comparisons involving a failed block use only the blocks completed by both conditions, so these are conditional-on-completion estimates. The bootstrap seed is fixed. These nominal, conditional confidence intervals summarize uncertainty in the condition mean under the working resampling assumption that completed block vectors are independent, identically distributed replicates. All blocks belong to one deployment campaign; their temporal dependence and the evolving storage history are not estimated away by resampling. They are not prediction intervals for individual runs, do not have proven exact finite-sample coverage here, and do not account for every systematic error or hardware population.

These are descriptive effect estimates, not a collection of significance tests with familywise error claims. We specify no p -value threshold, do not stop when an interval becomes favorable, and do not pool individual messages as independent replications. Recovery tables report all ten values through medians and ranges rather than presenting a high percentile from ten failure episodes as a precise tail estimate. No finite trial count can prove absence of all possible losses.

Additional post-observation diagnostics assess sensitivity without changing the primary dataset: leave-one-trial-out means for publication rates and within-trial p99 values; leave-one-common-block-out ratios for all thirteen paired effects; first-five versus last-five block summaries; and summaries between recorded execution interruptions. Leave-one-out ranges measure influence, not confidence coverage. Early/late contrasts and rank correlations with block order are descriptive, with no trend significance test or causal interpretation. The interruption segments follow planned trial ranges D001–D021, D022–D030, D031–D037, D038–D042, and D043–D060, using completed primary observations only. Their unequal, small cell counts prevent treating these segments as independent controlled replications. All computations and membership lists are retained.

2.1 Protocol development and execution incidents

The local protocol was written before main collection but was not externally preregistered. Twelve initial pilot trials were excluded. Before main collection, we added receive-request timestamps, increased the backlog from 4096 to 16384 messages after the pilot revealed short high-concurrency runs, and added the five-second primary durability series in place of relying on prefill timings. A final twelve-trial pilot verified the revised harness. Both pilot datasets and the prospective protocol versions are retained. The execution log records these changes; they are not presented as an immutable external registration.

During collection, a stream-creation request exceeded the five-second client deadline before a measured interval began. The server logged an 8.0186-second stream-creation handler. We retained that failed setup and the original logs, added checkpointed continuation, and retried the condition with unchanged settings and a fresh stream name. No completed measurement was replaced. After trial 88, a Redis-always warm-up returned through an error path and its cleanup also timed out, masking the primary operation error. Its original stack and log are retained. Administrative create, inventory, and deletion timeout settings were subsequently increased to 30 s. Redis default publication and acknowledgment socket timeouts and NATS response waits remained 5 s; the blocking-read overrides documented in Section 6 were unchanged. The durability series also stopped after 21 completed trials when a NATS-always warm-up publication timed

out; the server logged approximately 4.9925 s in deletion/read-loop processing for that warm-up stream. No timed sample for its next condition had begun. We preserved the original log and all completed samples, then added append-only continuation to that series without changing its measurement path or deadlines. A further NATS-always warm-up publication timed out after 30 completed durability trials, with a corresponding 3.5595 s deletion/read-loop warning; the existing checkpointed binary then continued without a further code change. The causes of these interruptions are not established. Setup and warm-up availability are therefore reported separately from timed outcomes.

Planned timed-publication trial D038 (Redis-always) subsequently aborted with a socket read timeout. Its partial client timings and confirmed-request count were lost on process abort and cannot be reconstructed from the retained evidence. A later retained-state snapshot contained 1568 distinct application IDs and zero pending consumer acknowledgments; this cannot identify which publications the client had confirmed. We preserve the failure as the planned outcome and exclude its unavailable throughput and latency from completed-run estimates. An orchestration error during failure-ledger copying caused an unintended completed replay of D038; that replay is retained but excluded from primary statistics. Stopping that continuation also caused an operator interruption of D043, whose partial broker state was preserved and whose planned condition was rerun after cleanup. These operational deviations and the exact evidence are recorded in the artifact. No observed slow completed planned trial was removed. A plot of publication throughput by block was added as a descriptive diagnostic after observing this variability; it is not a preregistered hypothesis test.

Table 1 consolidates the 550 planned outcomes and six additional documented attempts. Its accompanying record-level inventory links each entry to retained evidence and leaves unavailable times and confirmation counts missing. Successful warm-ups are part of their completed attempts, not additional rows. The two excluded twelve-trial pilots and infrastructure provisioning are outside this inventory. This is retrospective accounting of known execution outcomes, not a prospective exclusion rule or a denominator for estimating setup availability or total workload-completion cost. In particular, the later successful D043 attempt cannot reconstruct the interrupted attempt’s client history.

Retained outcome	Count	Treatment in this study
Completed timing trials	419	300 drain, 59 publication, 60 serial
Active consumer recovery	120	Recovered identifiers and cleared pending state
No-reclaimer controls	10	Pending at reconciliation; nominal budgets only
Planned publication failure	1	D038; client timings unavailable
Additional setup failure	1	Tmain052; excluded from timed estimates
Additional warm-up failures	3	Tmain089, D022, D031; timed trials not begun
Operator interruption	1	D043; interrupted client trace unavailable
Completed diagnostic replay	1	D038; retained and excluded from estimates

Table 1: Retained campaign outcomes and additional documented attempts. The first four rows account for all 550 planned outcomes. The remaining six attempts are separate from those outcomes. Counts do not reconstruct missing durations or establish end-to-end availability.

3 Original campaign results

3.1 Recorded outcomes and evidence quality

All 300 planned backlog-drain trials completed and their 4,915,200 message records passed complete identifier-set, uniqueness, retained-count, and pending-state checks. The 60 serial sensitivity trials similarly reconciled 61,440 messages. The primary publication series has 59 completed planned trials with 12,434,648 confirmed appends and one retained Redis-always timeout. Its accidental completed replay is excluded as specified in Section 2. Publication-only integrity checks establish the retained count, not a complete payload readback. The analyzer reconstructs each reported throughput and percentile from raw timings and rejects inconsistent summaries or a missing failure ledger.

All 120 active recovery episodes returned the original identifier, acknowledged it, and left one retained message with zero pending acknowledgments; JetStream reported two deliveries in each episode. All ten Redis no-reclaimer controls remained pending at the end of their observation windows. Every victim exit was recorded as killed by a signal. The largest kill-invocation-to-observed-exit bracket was 2.291 ms. The final seven-pod snapshot reported zero container restarts and zero cgroup OOM kills. Recorded driver CPU-counter differences showed no throttling within completed timed trials; this does not establish an idle testbed or identify the cause of the recorded timeouts.

3.2 RQ1: failure age, reclamation, and observed redelivery

Δ	α	System / sleep	$\Delta(1 - \alpha)$ (ms)	Median (ms)	Range (ms)
250	0.1	JetStream	225.0	226.5	[225.9, 226.9]
250	0.1	Redis / 10 ms	225.0	230.2	[228.4, 234.3]
250	0.1	Redis / 100 ms	225.0	265.4	[234.0, 313.9]
250	0.75	JetStream	62.5	63.6	[63.1, 64.5]
250	0.75	Redis / 10 ms	62.5	67.9	[63.9, 70.6]
250	0.75	Redis / 100 ms	62.5	134.0	[75.8, 161.2]
1000	0.1	JetStream	900.0	907.1	[905.4, 910.2]
1000	0.1	Redis / 10 ms	900.0	903.5	[900.4, 913.7]
1000	0.1	Redis / 100 ms	900.0	943.5	[911.7, 981.1]
1000	0.75	JetStream	250.0	257.9	[255.5, 260.6]
1000	0.75	Redis / 10 ms	250.0	255.4	[251.2, 269.4]
1000	0.75	Redis / 100 ms	250.0	319.4	[258.9, 339.4]

Table 2: Failure-to-redelivery latency for all recovery configurations. Each row contains ten killed-consumer episodes. $\Delta(1 - \alpha)$ is a nominal reference based on the scheduled age after client receipt, not a verified server-side lower bound. Ranges contain the minimum and maximum observed episode.

Table 2 and Figure 1 report shorter failure-to-redelivery intervals after later kills at the same configured timeout. At $\Delta = 250$ ms, the nominal remaining intervals are 225 and 62.5 ms. JetStream’s corresponding medians are 226.5 and 63.6 ms; Redis with a 10 ms reclaim sleep yields 230.2 and 67.9 ms. At $\Delta = 1000$ ms, the medians decrease from 907.1 to 257.9 ms for JetStream and from 903.5 to 255.4 ms for Redis with the shorter sleep. This direction is consistent with

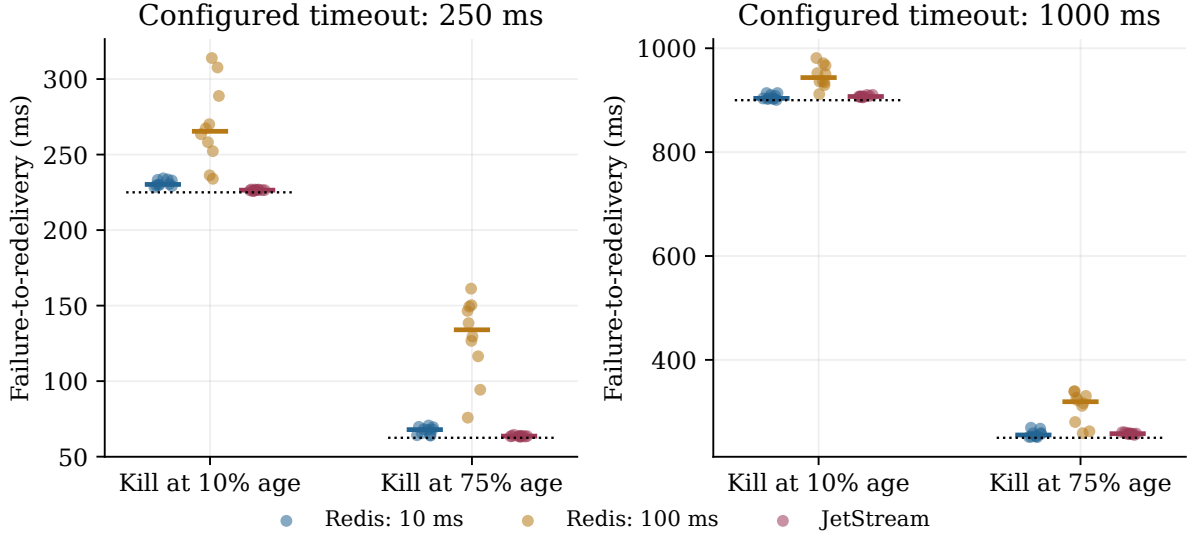


Figure 1: Every active-reclamation recovery observation. Horizontal colored marks show medians; horizontal jitter separates points and has no numerical meaning. Dotted lines show the nominal client-receipt-based residual-time reference. The ten no-reclaimer controls are censored and are reported separately rather than plotted as exact recovery times.

the residual-time decomposition in Theorem 1.1: later failure leaves less of the existing eligibility interval to wait out. Eligibility depends on message age and acknowledgment state; it can arise while a consumer remains alive. These observations do not establish a death-detection mechanism or directly observe the broker-side timer origin.

Redis reclamation frequency also matters. With a 100 ms sleep, the late-failure median rises to 134.0 ms at the 250 ms timeout and 319.4 ms at the 1000 ms timeout. Across the four age/timeout cells, the median observed excess $(\hat{r} - \hat{d}) - \Delta$ ranges from 4.0 to 6.2 ms with a 10 ms sleep, compared with 40.7–72.2 ms with a 100 ms sleep. JetStream’s median excess ranges from 1.9 to 8.6 ms. Neither a configured sleep nor an observed maximum is a proven future scheduling bound. JetStream’s 1000 ms medians are slightly above the shorter-sleep Redis medians, so the results do not support an unconditional claim that one system recovers faster.

The negative controls isolate a separate mechanism: ten observed non-recoveries with pending state intact are consistent with Proposition 1.3. Their finite observation procedures do not empirically prove infinite non-delivery; the proposition supplies that conditional counterexample from the stated new-read-only policy. Because exact censor endpoints were not recorded, these controls contribute descriptive evidence rather than exposure-time or survival estimates. The answer to RQ1 is therefore twofold: a failure leaves a residual timeout, and actual recovery additionally depends on reclamation or expiry selection and available demand. Under the active policies studied, all episodes recovered; without Redis reclamation, none of the controls did during observation.

3.3 RQ2: concurrency gains and cycle-latency cost

Increasing C from one to sixteen improved mean drain throughput in every profile (Table 3). Redis gains were 12.59, 11.02, and 8.59 times for memory, periodic, and always modes; JetStream

System	Mode	X_1 (kmsg/s)	X_{16} (kmsg/s)	X_{16}/X_1
Redis	memory	2.89 [2.74, 3.01]	36.37 [35.94, 36.83]	12.59 [12.13, 13.15]
Redis	periodic	2.66 [2.56, 2.76]	29.32 [27.73, 30.50]	11.02 [10.22, 11.79]
Redis	always	0.29 [0.29, 0.30]	2.51 [2.47, 2.55]	8.59 [8.47, 8.72]
JetStream	memory	2.30 [2.26, 2.34]	29.79 [29.43, 30.12]	12.94 [12.69, 13.23]
JetStream	periodic	2.27 [2.24, 2.29]	28.03 [26.89, 28.97]	12.36 [11.79, 12.83]
JetStream	always	2.24 [2.21, 2.27]	27.43 [27.04, 27.83]	12.24 [11.98, 12.54]

Table 3: Acknowledged backlog-drain throughput at one and sixteen consumers, with means and nominal 95% bootstrap intervals. Speedup is the ratio of condition means with a paired block-bootstrap interval. Each condition contains ten trials.

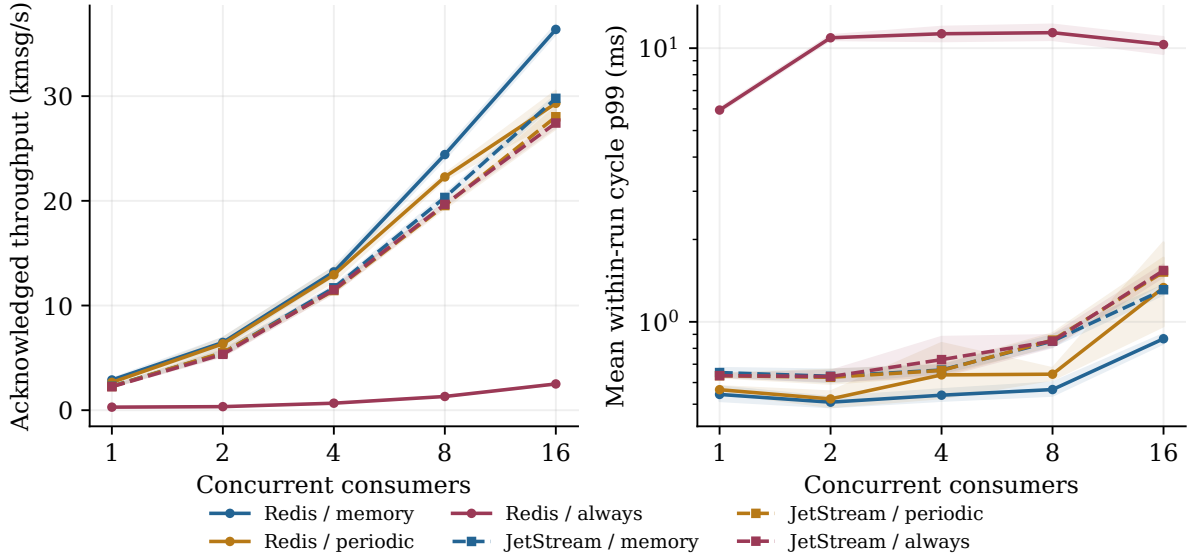


Figure 2: The complete concurrency response. Lines connect measured consumer counts without estimating unmeasured counts. Shaded bands are nominal 95% bootstrap confidence intervals for condition means; the right panel uses a logarithmic latency axis. Cycle latency is successful receive-request invocation to acknowledgment completion.

gains were 12.94, 12.36, and 12.24 times. All six endpoint gains are below the sixteenfold increase in workers. The memory profiles reached 36.37 kmsg/s for Redis and 29.79 kmsg/s for JetStream at sixteen consumers. Periodic profiles reached 29.32 and 28.03 kmsg/s. These rates describe the measured request/acknowledgment path with a prefilled backlog, not a server-capacity ceiling or an open-loop saturation point.

The throughput gain has a cycle-latency cost. From one to sixteen consumers, mean within-trial cycle p99 increased from 0.543 to 0.867 ms for Redis memory and from 0.653 to 1.310 ms for JetStream memory. Redis periodic increased from 0.565 to 1.334 ms and JetStream periodic from 0.636 to 1.520 ms. Redis always increased from 5.942 to 10.311 ms, while JetStream always increased from 0.636 to 1.541 ms. In contrast, the corresponding common-start-to-receipt drain p99 values decrease between these endpoints (the complete matrix below): more workers complete the backlog sooner despite a slower individual cycle.

Reconstructed occupancy U lies between 0.9936 and 0.9995 across all 300 trials. Every worker’s recorded intervals are nonoverlapping and inside the measured window, satisfying the checked premises of Theorem 1.5. Thus recorded successful cycles occupy nearly all available worker time. The identity $X = CU/\bar{R}$ explains how increased mean cycle cost offsets the added concurrency; it does not identify which internal contention mechanism caused that cost. Nor does a p99 value replace $\bar{R}$ in this identity.

The always-mode endpoint is particularly easy to misread: JetStream drains 27.43 kmsg/s and Redis 2.51 kmsg/s, but their consumer-state synchronization contracts differ. RQ2 is answered by sublinear endpoint scaling with rising cycle tails within each profile. A cross-system ranking under an equal synchronous consumer-progress durability guarantee is not established by these configurations.

3.4 RQ3: confirmed-publication cost under persistence policies

System	Mode	n	Throughput (kmsg/s)	p50 (ms)	p99 (ms)
Redis	memory	10	73.90 [73.60, 74.19]	0.204	0.474 [0.466, 0.486]
Redis	periodic	10	61.38 [61.08, 61.67]	0.247	0.557 [0.551, 0.565]
Redis	always	9	3.03 [1.73, 4.31]	9.959	29.001 [11.615, 50.929]
JetStream	memory	10	60.38 [58.99, 61.32]	0.241	0.704 [0.678, 0.742]
JetStream	periodic	10	49.49 [48.29, 50.48]	0.281	0.849 [0.832, 0.869]
JetStream	always	10	0.37 [0.19, 0.56]	108.664	579.568 [91.362, 1,459.647]

Table 4: Five-second publication workload with sixteen independent publishers. Throughput and p99 entries give means and nominal 95% bootstrap intervals; p50 entries are means of within-trial medians. n is the number of completed planned trials out of ten; the failed Redis-always trial is reported separately.

The periodic configurations exhibited lower confirmed-publication throughput than the memory configurations in this campaign. Memory and periodic means were 73.90 and 61.38 kmsg/s for Redis, and 60.38 and 49.49 kmsg/s for JetStream. The paired periodic/memory throughput ratios were 0.8305 [0.8237, 0.8371] for Redis and 0.8197 [0.7933, 0.8511] for JetStream, using nominal 95% block-bootstrap intervals. Mean within-trial publication p99 was 0.474 versus 0.557 ms and 0.704 versus 0.849 ms, respectively. These descriptive differences include the evolving storage history and shared environment; they do not isolate the causal cost of synchronization policy or measure power-loss data loss.

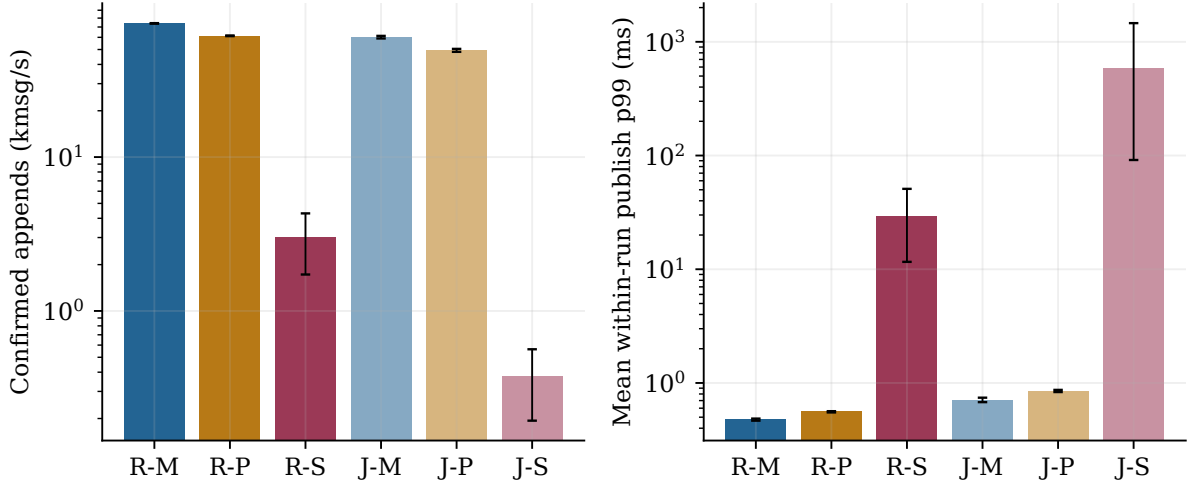


Figure 3: Confirmed-publication throughput and latency for Redis (R) and JetStream (J), in memory (M), periodic (P), and always (S) profiles. Both vertical axes are logarithmic. Error bars give nominal 95% bootstrap confidence intervals for means conditional on completion (nine Redis-always trials and ten for each other profile).

Always-mode publication was substantially slower and more variable. The nine completed planned Redis-always trials averaged 3028 msg/s [1725, 4306], with a mean within-trial p99 of 29.0 ms [11.6, 50.9]. The tenth planned trial timed out at the measured request deadline and has no reconstructible client latency sample. The ten completed JetStream-always trials averaged 375 msg/s [194, 564], with a mean within-trial p99 of 579.6 ms [91.4, 1459.6]. The wide intervals and per-block observations in Figure 4 are material results: the point estimates should not be presented as stable production tail-latency guarantees. The separately recorded setup and warm-up incidents further constrain operational interpretation.

System	Mode	Mean p99 (ms)	Median p99 (ms)	LOO means (ms)	Early / late rate (kmsg/s)
Redis	memory	0.47	0.47	[0.47, 0.48]	73.98 / 73.83
Redis	periodic	0.56	0.56	[0.56, 0.56]	61.38 / 61.37
Redis	always	29.00	7.46	[20.15, 32.10]	3.78 / 2.08
JetStream	memory	0.70	0.68	[0.69, 0.71]	59.68 / 61.07
JetStream	periodic	0.85	0.84	[0.84, 0.85]	50.35 / 48.64
JetStream	always	579.57	160.26	[150.53, 639.37]	0.45 / 0.30

Table 5: Post-observation publication sensitivity. The mean and median summarize completed trials’ p99 values. LOO gives the range of means after omitting each completed trial in turn, solely as an influence diagnostic. Early and late rates use blocks 1–5 and 6–10. Redis-always has five early and four late completions; other cells have five in each half. No observation is removed from primary estimates.

Table 5 shows the sensitivity of always-mode summaries. JetStream-always’s mean within-trial p99 is 579.6 ms, whereas the median of those ten p99 values is 160.3 ms. Its leave-one-out means range from 150.5 to 639.4 ms; the minimum omits D040, whose within-trial p99 is 4440.9 ms.

Redis-always’s corresponding mean and median are 29.0 and 7.5 ms. These contrasts characterize influence and skew, not grounds for discarding slow observations.

System	Mode	Trials	Messages per trial	Positions above p99
Redis	memory	10	365,505–372,128	3,655–3,721
Redis	periodic	10	303,669–310,204	3,036–3,102
Redis	always	9	2,608–26,221	26–262
JetStream	memory	10	273,779–310,872	2,737–3,108
JetStream	periodic	10	243,055–257,750	2,430–2,577
JetStream	always	10	149–3,577	1–35

Table 6: Sample information underlying completed publication trials’ empirical p99 values. Ranges span the retained trials of each profile. Positions above p99 are $N - \lceil 0.99N \rceil$ and do not imply independent tail observations.

Table 6 makes the unequal sample sizes explicit. All 59 trial counts, ranks, and p99 values are retained in `data/derived/publication-tail-samples.csv`. D040 contains 149 confirmed publications, so its p99 is order statistic 148, the second-largest observation. Its influential tail estimate consequently has very little information above the selected rank. The mean of trial p99 values remains a descriptive summary of these observed trials; it is weak evidence for a stable population tail, and equal trial weights do not equalize percentile precision. Longer, independently repeated observations would be required for a stronger tail characterization. We retain D040 and do not replace its percentile with a pooled-message estimate.

The direction of all thirteen reported paired contrasts survives every leave-one-common-block-out calculation. Magnitudes remain conditional on the retained blocks: the periodic/memory publication ratios range from 0.8285 to 0.8320 for Redis and from 0.8076 to 0.8282 for JetStream in that diagnostic. Early versus late always-mode throughput declines from 3.78 to 2.08 kmsg/s for Redis and from 0.45 to 0.30 kmsg/s for JetStream. A simple monotonic degradation model is also insufficient: JetStream-always averages 483.3 msg/s in D001–D021, 24.6 msg/s in the interruption-bounded D038–D042 segment containing its single D040 observation, and 462.7 msg/s in D043–D060. The artifact reports every segment and cell count. These uncontrolled contrasts establish variability and sensitivity to execution history, not that an interruption or AOF growth caused a particular outcome. Independent campaign repetitions are needed to assess repeatability beyond this deployment.

Against memory mode, the paired always/memory throughput ratio is 0.0410 [0.0233, 0.0583] for Redis over nine common completed blocks and 0.00621 [0.00318, 0.00945] for JetStream over ten blocks. Their reciprocal point estimates are 24.4- and 161.1-fold slowdowns. Because the Redis estimate excludes an unavailable failed-trial metric, it describes successful runs and can understate the operational cost of the complete outcome distribution. We neither impute zero throughput nor replace the failed outcome with its diagnostic replay.

The original serial-publisher sensitivity gives a different view (the serial table below): always-mode publication averaged 603 msg/s [591, 614] for Redis and 585 msg/s [565, 598] for JetStream. The close point estimates do not establish equivalence. This shorter fixed-count workload differs in both publication concurrency and duration, so it cannot isolate batching as the cause of the contrast with the sixteen-publisher results. No syscall or storage-device trace was collected in that original campaign to establish the mechanism. The follow-up’s matched-duration publication and syscall diagnostics are a separate dataset.

RQ3 is therefore answered as a descriptive, workload-specific tradeoff: the configured periodic profiles preserve most memory-mode throughput with modestly higher observed tails, whereas always-mode confirmed publication incurs much larger and more variable observed costs in this shared Btrfs/NVMe environment. The synchronization contract, accumulated storage history, completion conditioning, and failure class must accompany that performance statement. Neither these append measurements nor the fast JetStream drain acknowledgments prove equivalent durable consumer progress or stable-media survival after a power cut.

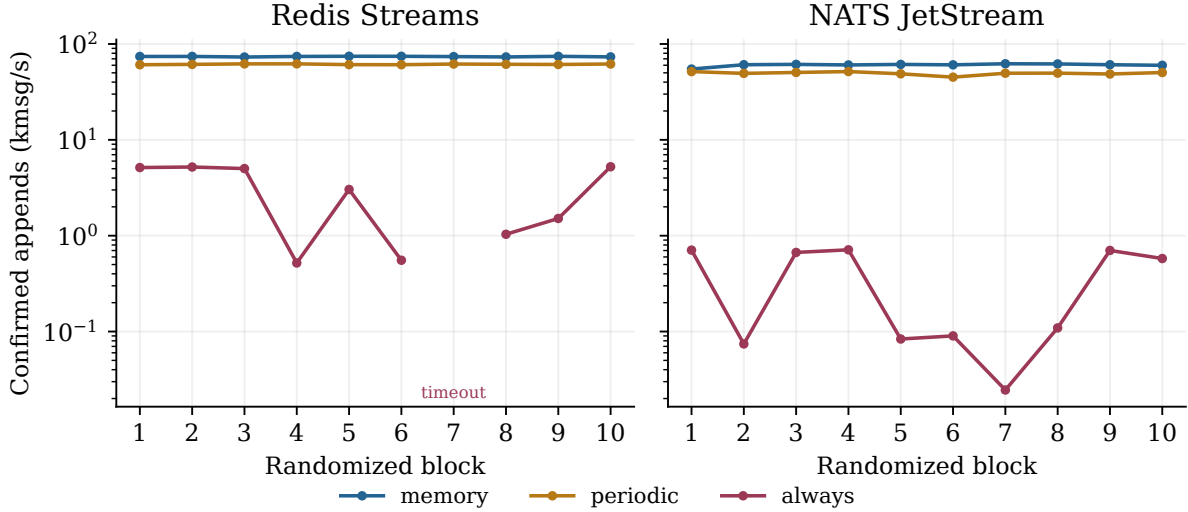


Figure 4: Descriptive publication-throughput diagnostic by randomized block. Each point is one completed planned trial. The timeout annotation has no throughput coordinate and identifies the failed Redis-always trial in block seven. A missing value interrupts its line; no failure is imputed as zero. Blocks are ordered experimental groups, not equal wall-time intervals.

4 Complete original concurrency matrix

System	Mode	C	Throughput (kmsg/s)	Cycle p99 (ms)	Drain p99 (ms)
Redis	memory	1	2.89 [2.74, 3.01]	0.543	5649.9
Redis	memory	2	6.47 [6.06, 6.90]	0.509	2532.3
Redis	memory	4	13.21 [12.82, 13.64]	0.539	1231.2
Redis	memory	8	24.42 [23.98, 24.83]	0.566	664.4
Redis	memory	16	36.37 [35.94, 36.83]	0.867	445.8
Redis	periodic	1	2.66 [2.56, 2.76]	0.565	6116.9
Redis	periodic	2	6.34 [5.81, 6.90]	0.523	2606.0
Redis	periodic	4	12.94 [12.31, 13.62]	0.640	1261.8
Redis	periodic	8	22.28 [22.02, 22.55]	0.644	728.0
Redis	periodic	16	29.32 [27.73, 30.50]	1.334	557.6
Redis	always	1	0.29 [0.29, 0.30]	5.942	55604.9
Redis	always	2	0.34 [0.33, 0.35]	10.915	47736.5
Redis	always	4	0.67 [0.66, 0.68]	11.291	24240.7
Redis	always	8	1.31 [1.30, 1.33]	11.395	12380.0

System	Mode	C	Throughput (kmsg/s)	Cycle p99 (ms)	Drain p99 (ms)
Redis	always	16	2.51 [2.47, 2.55]	10.311	6469.7
JetStream	memory	1	2.30 [2.26, 2.34]	0.653	7052.2
JetStream	memory	2	5.46 [5.25, 5.74]	0.633	2986.6
JetStream	memory	4	11.69 [11.52, 11.90]	0.667	1388.1
JetStream	memory	8	20.32 [20.05, 20.61]	0.851	798.4
JetStream	memory	16	29.79 [29.43, 30.12]	1.310	544.3
JetStream	periodic	1	2.27 [2.24, 2.29]	0.636	7155.0
JetStream	periodic	2	5.50 [5.20, 5.84]	0.627	2970.8
JetStream	periodic	4	11.43 [11.23, 11.65]	0.664	1421.1
JetStream	periodic	8	19.55 [19.28, 19.80]	0.857	829.6
JetStream	periodic	16	28.03 [26.89, 28.97]	1.520	580.6
JetStream	always	1	2.24 [2.21, 2.27]	0.636	7245.0
JetStream	always	2	5.34 [5.15, 5.61]	0.631	3051.9
JetStream	always	4	11.50 [11.20, 11.79]	0.728	1413.7
JetStream	always	8	19.66 [19.49, 19.81]	0.850	824.5
JetStream	always	16	27.43 [27.04, 27.83]	1.541	591.9

5 Original serial-publisher sensitivity

System	Mode	Throughput (msg/s)	Publish p99 (ms)
Redis	memory	5,946.8 [5,385.6, 6,724.1]	0.240 [0.217, 0.257]
Redis	periodic	5,494.4 [5,076.8, 6,107.2]	0.257 [0.233, 0.278]
Redis	always	602.7 [591.3, 613.6]	2.847 [2.365, 3.340]
JetStream	memory	4,736.8 [4,479.8, 5,070.7]	0.325 [0.304, 0.351]
JetStream	periodic	4,911.5 [4,457.7, 5,425.8]	0.334 [0.304, 0.362]
JetStream	always	584.8 [564.8, 598.0]	2.756 [2.063, 3.807]

Table 8: Original fixed-count, one-publisher sensitivity.

6 Client waits and observation conventions

All request timestamps use the driver’s Linux monotonic clock. Broker telemetry uses its own node clock. Setup, warm-up, trace serialization and stored-payload readback are outside timing. Original ordinary reads and recovery survivor polls use a 100 ms Redis block or JetStream fetch wait. Follow-up drain requests and victim initial reads use five seconds. The pinned Redis client adds ten seconds to positive typed blocking-read socket deadlines, producing 10.1 s or 15 s, respectively [4]; the raw CLAIM command instead retains the default five-second socket timeout. JetStream’s fetch-request expiry is approximately 90% of the client wait and can leave a pull-demand gap [6]. Redis publication and acknowledgment use five-second socket-stage read/write limits, not a whole-call deadline; NATS uses five-second response waits. The original drain loop additionally checks a 120 s budget, while the external follow-up invocation has a separate 600 s controller limit. None of these waits is the broker eligibility threshold.

7 Follow-up condition summaries

The following tables concern the separately collected fresh-store follow-up. Runs 01, 02 and 03 identify separate deployments on the same hardware. Cross-run point summaries equally weight the available run estimates. Bracketed ranges contain their minimum and maximum; they are not confidence or prediction intervals. M, P and S denote memory, periodic and always append profiles. Publication P is the number of publishers, whereas drain C is the number of consumers. A failed outcome is absent from successful-run numerical means but remains in the outcome and pair counts.

7.1 Publication means and tail sample information

Rates are in kmsg/s. Trial p99 is the equally weighted mean of within-trial nearest-rank percentiles inside each run, followed by equal weighting across available run means. n is completed/planned trials. The range of sample counts is reported by run below; every trial's count, selected p99 rank and number of positions above that rank are in the accompanying `followup/derived/trials.csv`.

System	Mode	P	n	Rate, kmsg/s	Trial p99, ms
Redis 7.4.2	M	1	9/9	5.73 [5.50, 5.86]	0.26 [0.25, 0.26]
Redis 7.4.2	M	16	9/9	72.13 [71.34, 72.62]	0.49 [0.49, 0.50]
Redis 7.4.2	P	1	9/9	5.26 [4.87, 5.55]	0.27 [0.26, 0.29]
Redis 7.4.2	P	16	9/9	60.70 [60.20, 61.21]	0.56 [0.56, 0.56]
Redis 7.4.2	S	1	7/9	0.40 [0.35, 0.42]	16.73 [13.95, 20.20]
Redis 7.4.2	S	16	7/9	4.51 [3.83, 5.00]	11.78 [6.16, 18.90]
Redis 8.10.2	M	1	9/9	5.50 [5.48, 5.52]	0.26 [0.26, 0.26]
Redis 8.10.2	M	16	9/9	73.26 [72.80, 73.71]	0.47 [0.46, 0.48]
Redis 8.10.2	P	1	9/9	5.36 [5.30, 5.47]	0.27 [0.26, 0.27]
Redis 8.10.2	P	16	9/9	60.49 [59.57, 61.13]	0.56 [0.55, 0.58]
Redis 8.10.2	S	1	6/9	0.51 [0.35, 0.59]	7.72 [2.68, 17.13]
Redis 8.10.2	S	16	9/9	4.35 [3.46, 5.05]	16.98 [6.02, 29.12]
JetStream 2.10.24	M	1	9/9	4.61 [4.59, 4.61]	0.31 [0.31, 0.32]
JetStream 2.10.24	M	16	9/9	60.47 [59.79, 61.13]	0.69 [0.67, 0.71]
JetStream 2.10.24	P	1	8/9	4.19 [4.11, 4.29]	0.35 [0.34, 0.36]
JetStream 2.10.24	P	16	9/9	49.16 [47.77, 50.34]	0.90 [0.86, 0.94]
JetStream 2.10.24	S	1	9/9	0.44 [0.36, 0.54]	13.38 [4.40, 21.93]
JetStream 2.10.24	S	16	8/9	0.55 [0.46, 0.63]	105.96 [48.88, 165.48]
JetStream 2.15.0	M	1	9/9	4.48 [4.47, 4.50]	0.32 [0.32, 0.32]
JetStream 2.15.0	M	16	9/9	60.62 [60.36, 60.84]	0.64 [0.63, 0.65]
JetStream 2.15.0	P	1	9/9	4.22 [4.20, 4.23]	0.34 [0.33, 0.34]
JetStream 2.15.0	P	16	9/9	49.75 [49.09, 50.09]	0.80 [0.78, 0.82]
JetStream 2.15.0	S	1	8/9	0.46 [0.41, 0.57]	12.07 [2.78, 22.97]
JetStream 2.15.0	S	16	8/9	0.56 [0.42, 0.68]	295.64 [45.58, 725.22]

7.2 Publication by deployment

Each condition has three planned trials within a deployment. Columns report the condition's arithmetic mean throughput and mean empirical trial p99, conditional on completion. Count range is the minimum/maximum number of confirmed publications among those completed trials.

Run	System	Mode	P	n	kmsg/s	p99, ms	Count range
01	Redis 7.4.2	M	1	3/3	5.86	0.26	86617–89404
02	Redis 7.4.2	M	1	3/3	5.83	0.25	86695–88585

Run	System	Mode	P	n	kmsg/s	p99, ms	Count range
03	Redis 7.4.2	M	1	3/3	5.50	0.26	73168–87462
01	Redis 7.4.2	M	16	3/3	72.42	0.49	1063608–1103486
02	Redis 7.4.2	M	16	3/3	71.34	0.50	1009276–1102191
03	Redis 7.4.2	M	16	3/3	72.62	0.49	1069841–1101776
01	Redis 7.4.2	P	1	3/3	5.55	0.26	82805–83646
02	Redis 7.4.2	P	1	3/3	5.37	0.27	73441–84247
03	Redis 7.4.2	P	1	3/3	4.87	0.29	72885–73175
01	Redis 7.4.2	P	16	3/3	60.20	0.56	875200–924144
02	Redis 7.4.2	P	16	3/3	60.69	0.56	887260–927302
03	Redis 7.4.2	P	16	3/3	61.21	0.56	897989–933609
01	Redis 7.4.2	S	1	3/3	0.42	13.95	886–9143
02	Redis 7.4.2	S	1	2/3	0.35	20.20	1761–8773
03	Redis 7.4.2	S	1	2/3	0.42	16.05	3585–8903
01	Redis 7.4.2	S	16	2/3	3.83	18.90	37311–77608
02	Redis 7.4.2	S	16	2/3	5.00	6.16	73784–76364
03	Redis 7.4.2	S	16	3/3	4.69	10.29	59595–76581
01	Redis 8.10.2	M	1	3/3	5.52	0.26	74269–87818
02	Redis 8.10.2	M	1	3/3	5.48	0.26	73464–87073
03	Redis 8.10.2	M	1	3/3	5.50	0.26	73999–87022
01	Redis 8.10.2	M	16	3/3	72.80	0.48	1084779–1101455
02	Redis 8.10.2	M	16	3/3	73.71	0.46	1099413–1109992
03	Redis 8.10.2	M	16	3/3	73.28	0.46	1095054–1105853
01	Redis 8.10.2	P	1	3/3	5.30	0.27	72271–84374
02	Redis 8.10.2	P	1	3/3	5.32	0.27	73471–83211
03	Redis 8.10.2	P	1	3/3	5.47	0.26	81896–82151
01	Redis 8.10.2	P	16	3/3	61.13	0.55	910897–923491
02	Redis 8.10.2	P	16	3/3	59.57	0.58	847181–922708
03	Redis 8.10.2	P	16	3/3	60.76	0.55	895930–926585
01	Redis 8.10.2	S	1	2/3	0.59	3.35	8675–8929
02	Redis 8.10.2	S	1	2/3	0.59	2.68	8815–8870
03	Redis 8.10.2	S	1	2/3	0.35	17.13	1685–8858
01	Redis 8.10.2	S	16	3/3	5.05	6.02	73322–77497
02	Redis 8.10.2	S	16	3/3	3.46	29.12	30549–76566
03	Redis 8.10.2	S	16	3/3	4.52	15.81	59508–76001
01	JetStream 2.10.24	M	1	3/3	4.59	0.32	68143–69356
02	JetStream 2.10.24	M	1	3/3	4.61	0.31	68542–70456
03	JetStream 2.10.24	M	1	3/3	4.61	0.31	68729–69462
01	JetStream 2.10.24	M	16	3/3	59.79	0.71	886488–909818
02	JetStream 2.10.24	M	16	3/3	60.51	0.69	894937–928256
03	JetStream 2.10.24	M	16	3/3	61.13	0.67	914429–920304
01	JetStream 2.10.24	P	1	3/3	4.11	0.36	58828–63146
02	JetStream 2.10.24	P	1	3/3	4.29	0.34	63721–65551
03	JetStream 2.10.24	P	1	2/3	4.17	0.35	61715–63332
01	JetStream 2.10.24	P	16	3/3	49.36	0.88	733595–745935
02	JetStream 2.10.24	P	16	3/3	47.77	0.94	677445–738843
03	JetStream 2.10.24	P	16	3/3	50.34	0.86	747495–764302
01	JetStream 2.10.24	S	1	3/3	0.42	13.81	1510–8602
02	JetStream 2.10.24	S	1	3/3	0.36	21.93	1022–8572
03	JetStream 2.10.24	S	1	3/3	0.54	4.40	7674–8753
01	JetStream 2.10.24	S	16	3/3	0.55	103.53	3941–10488
02	JetStream 2.10.24	S	16	2/3	0.46	165.48	3535–10366
03	JetStream 2.10.24	S	16	3/3	0.63	48.88	8839–10358
01	JetStream 2.15.0	M	1	3/3	4.47	0.32	64320–68841
02	JetStream 2.15.0	M	1	3/3	4.47	0.32	64159–69257
03	JetStream 2.15.0	M	1	3/3	4.50	0.32	64403–69803
01	JetStream 2.15.0	M	16	3/3	60.84	0.64	906793–918179

Run	System	Mode	P	n	kmsg/s	p99, ms	Count range
02	JetStream 2.15.0	M	16	3/3	60.36	0.65	876947–928091
03	JetStream 2.15.0	M	16	3/3	60.67	0.63	905314–917583
01	JetStream 2.15.0	P	1	3/3	4.20	0.34	62006–63665
02	JetStream 2.15.0	P	1	3/3	4.23	0.33	62657–64606
03	JetStream 2.15.0	P	1	3/3	4.23	0.34	62445–64079
01	JetStream 2.15.0	P	16	3/3	50.09	0.79	738387–768069
02	JetStream 2.15.0	P	16	3/3	50.07	0.78	744559–755619
03	JetStream 2.15.0	P	16	3/3	49.09	0.82	717729–764826
01	JetStream 2.15.0	S	1	2/3	0.42	10.45	4135–8392
02	JetStream 2.15.0	S	1	3/3	0.41	22.97	1022–8614
03	JetStream 2.15.0	S	1	3/3	0.57	2.78	8546–8603
01	JetStream 2.15.0	S	16	2/3	0.68	45.58	10169–10236
02	JetStream 2.15.0	S	16	3/3	0.60	116.10	7895–10276
03	JetStream 2.15.0	S	16	3/3	0.42	725.22	976–10394

7.3 Drain by deployment

Each condition has two planned trials per deployment and 65536 messages per completed trial. CPU is the driver quota, with matching GOMAXPROCS. Driver and Broker columns give observed mean cores during their phase-bracketing samples, including other processes within the respective containers. The p99 column is the mean empirical request-to-acknowledgment cycle p99 in milliseconds.

Run	System	Mode	C	CPU	n	kmsg/s	p99, ms	Driver	Broker
01	Redis	M	1	2	2/2	2.70	0.53	0.33	0.08
02	Redis	M	1	2	2/2	2.90	0.50	0.36	0.08
03	Redis	M	1	2	2/2	2.90	0.51	0.36	0.09
01	Redis	M	16	2	2/2	34.74	0.90	1.07	0.56
02	Redis	M	16	2	2/2	35.36	0.88	1.08	0.56
03	Redis	M	16	2	2/2	34.85	0.90	1.08	0.56
01	Redis	M	1	4	2/2	2.93	0.51	0.35	0.08
02	Redis	M	1	4	2/2	2.69	0.53	0.34	0.08
03	Redis	M	1	4	2/2	2.92	0.50	0.36	0.08
01	Redis	M	16	4	2/2	35.77	0.81	1.42	0.56
02	Redis	M	16	4	2/2	36.39	0.76	1.44	0.56
03	Redis	M	16	4	2/2	35.99	0.82	1.43	0.56
01	Redis	P	1	2	2/2	2.45	0.56	0.32	0.12
02	Redis	P	1	2	2/2	2.59	0.54	0.33	0.12
03	Redis	P	1	2	2/2	2.73	0.53	0.35	0.13
01	Redis	P	16	2	2/2	28.69	1.04	0.98	0.65
02	Redis	P	16	2	2/2	28.46	1.04	0.97	0.64
03	Redis	P	16	2	2/2	28.51	1.04	0.98	0.64
01	Redis	P	1	4	2/2	2.45	0.56	0.31	0.12
02	Redis	P	1	4	2/2	2.60	0.54	0.33	0.12
03	Redis	P	1	4	2/2	2.61	0.54	0.33	0.12
01	Redis	P	16	4	2/2	28.49	0.94	1.31	0.65
02	Redis	P	16	4	2/2	29.84	0.90	1.32	0.64
03	Redis	P	16	4	2/2	29.59	0.98	1.33	0.63
01	JetStream	M	1	2	2/2	2.28	0.62	0.45	0.22
02	JetStream	M	1	2	2/2	2.29	0.62	0.45	0.22
03	JetStream	M	1	2	2/2	2.25	0.64	0.43	0.21
01	JetStream	M	16	2	2/2	29.65	1.24	1.11	1.30
02	JetStream	M	16	2	2/2	30.56	1.17	1.12	1.30
03	JetStream	M	16	2	2/2	28.41	1.42	1.10	1.27

Run	System	Mode	C	CPU	n	kmsg/s	p99, ms	Driver	Broker
01	JetStream	M	1	4	2/2	2.31	0.62	0.45	0.22
02	JetStream	M	1	4	2/2	2.30	0.63	0.45	0.22
03	JetStream	M	1	4	2/2	2.24	0.63	0.45	0.21
01	JetStream	M	16	4	2/2	32.71	0.97	1.50	1.36
02	JetStream	M	16	4	2/2	32.90	0.98	1.50	1.34
03	JetStream	M	16	4	2/2	32.40	1.01	1.48	1.34
01	JetStream	P	1	2	2/2	2.28	0.63	0.44	0.22
02	JetStream	P	1	2	2/2	2.27	0.64	0.44	0.22
03	JetStream	P	1	2	2/2	2.22	0.65	0.44	0.22
01	JetStream	P	16	2	2/2	28.78	1.32	1.11	1.29
02	JetStream	P	16	2	2/2	28.95	1.31	1.12	1.28
03	JetStream	P	16	2	2/2	28.37	1.42	1.10	1.27
01	JetStream	P	1	4	2/2	2.26	0.64	0.45	0.22
02	JetStream	P	1	4	2/2	2.28	0.64	0.45	0.22
03	JetStream	P	1	4	2/2	2.28	0.62	0.45	0.22
01	JetStream	P	16	4	2/2	31.67	1.07	1.47	1.35
02	JetStream	P	16	4	2/2	31.57	1.07	1.48	1.34
03	JetStream	P	16	4	2/2	31.61	1.09	1.46	1.33

7.4 Recovery excess and paired controls

Excess is redelivery receipt minus initial receipt minus the configured threshold. Each live/killed cell has two planned episodes in each deployment. Entries give equally weighted deployment means and their observed ranges. Killed minus live is calculated within paired blocks before averaging deployment estimates. R7/R8 denote Redis 7.4.2/8.10.2, auto denotes XAUTOCLAIM with a 10 ms sleep, and J denotes JetStream. Neither a small observed difference nor overlap establishes equivalence.

Policy	Δ	Age, %	Live excess, ms	Killed excess, ms	Killed – live, ms
R7 auto	250	10	6.38 [5.04, 8.82]	5.61 [0.98, 8.09]	-0.77 [-4.06, 2.48]
R8 auto	250	10	5.44 [2.70, 7.98]	6.05 [5.17, 6.99]	0.61 [-0.99, 3.28]
R8 CLAIM	250	10	80.02 [78.43, 81.20]	78.29 [74.45, 81.22]	-1.74 [-6.75, 2.78]
J2.10	250	10	2.12 [2.05, 2.25]	2.17 [2.09, 2.21]	0.05 [-0.16, 0.16]
J2.15	250	10	1.98 [1.62, 2.17]	1.94 [1.71, 2.22]	-0.04 [-0.47, 0.28]
R7 auto	250	75	5.11 [3.73, 7.13]	7.25 [3.30, 10.50]	2.14 [-1.17, 6.77]
R8 auto	250	75	6.66 [2.84, 9.21]	2.26 [1.60, 3.52]	-4.40 [-7.55, 0.68]
R8 CLAIM	250	75	72.12 [62.22, 78.60]	81.08 [76.86, 87.83]	8.96 [3.02, 14.64]
J2.10	250	75	2.02 [1.97, 2.10]	2.00 [1.95, 2.07]	-0.02 [-0.12, 0.09]
J2.15	250	75	1.93 [1.63, 2.23]	2.01 [1.90, 2.10]	0.08 [-0.33, 0.46]
R7 auto	1000	10	8.71 [8.13, 9.53]	4.66 [2.80, 6.13]	-4.05 [-6.72, -2.00]
R8 auto	1000	10	8.08 [6.00, 10.70]	6.30 [3.20, 10.52]	-1.78 [-5.52, 4.52]
R8 CLAIM	1000	10	35.63 [26.64, 53.40]	30.04 [26.49, 36.71]	-5.58 [-26.91, 9.87]
J2.10	1000	10	7.17 [6.28, 7.87]	8.57 [8.21, 8.81]	1.40 [0.34, 2.53]
J2.15	1000	10	6.15 [4.68, 7.08]	5.24 [4.52, 5.93]	-0.91 [-1.42, -0.16]
R7 auto	1000	75	6.26 [2.93, 9.41]	6.58 [5.02, 8.49]	0.32 [-0.92, 2.09]
R8 auto	1000	75	7.73 [7.03, 8.90]	7.27 [3.05, 9.70]	-0.46 [-3.97, 1.79]
R8 CLAIM	1000	75	31.44 [20.52, 49.34]	33.22 [28.09, 35.80]	1.78 [-13.54, 11.32]
J2.10	1000	75	8.92 [8.46, 9.80]	7.99 [7.53, 8.37]	-0.92 [-2.27, -0.09]
J2.15	1000	75	5.52 [5.28, 5.71]	5.75 [4.52, 6.59]	0.23 [-0.76, 1.03]

The plain-read controls below report actual reference-to-survivor-loop-completion durations. Their nominal observation budget starts after the reference/kill bracket and equals twice the threshold. A complete outcome means that the message remained pending without redelivery

through that procedure. These endpoints are available for the follow-up and are not reconstructed for the historical controls.

Trial	Redis	Δ , ms	Outcome	Observation, s
01/F0185	7.4.2	250	complete	0.509
01/F0186	7.4.2	250	complete	0.595
01/F0187	8.10.2	250	complete	0.699
01/F0188	8.10.2	250	complete	0.503
02/F0185	7.4.2	250	complete	0.507
02/F0186	7.4.2	250	complete	0.503
02/F0187	8.10.2	250	complete	0.503
02/F0188	8.10.2	250	complete	0.697
03/F0185	7.4.2	250	complete	0.511
03/F0186	7.4.2	250	complete	0.614
03/F0187	8.10.2	250	complete	0.511
03/F0188	8.10.2	250	complete	0.505

7.5 Compression-policy diagnostics

System	Payload	Inherited, kmsg/s	Off, kmsg/s	Off/inherited	$n_i/n_o/n_p$
Redis	Filler	5.05 [4.93, 5.18]	5.23 [5.12, 5.32]	1.04 [0.99, 1.06]	3/3/3
Redis	Template	5.16 [5.09, 5.25]	5.14 [5.12, 5.17]	1.00 [0.98, 1.02]	3/3/3
JetStream	Filler	0.67 [0.66, 0.68]	0.68 [0.67, 0.69]	1.02 [0.99, 1.04]	3/3/3
JetStream	Template	0.70 [0.66, 0.73]	0.69 [0.68, 0.70]	0.99 [0.94, 1.05]	3/3/3

Table 14: Current always-profile compression-policy sensitivity at sixteen publishers: equal-weight deployment means and ranges. Filler is the original compressible payload; Template is the fixed pseudorandom bank. Off/inherited is a matched throughput ratio; $n_i/n_o/n_p$ counts complete inherited/off trials/paired comparisons, each out of three. Both policies use the same Btrfs filesystem and NVMe device.

7.6 Failed follow-up outcomes

The table identifies each retained failed trial and its stage. Conf. and Unknown are client event classifications, not a broker-presence partition. Join is elapsed time from the timed phase’s start until workers joined, when available. Stored is the count at a later inventory query. Inventory queries are sequential, and timed-out operations can complete between them; these counts do not reconstruct the identities or confirmations of unknown calls. A dash marks an unavailable field, not zero. No failed observation is replayed into a planned success.

Trial	System	Mode	P	Stage	Conf.	Unknown	Join, s	Stored
01/F0006	J2.15.0	S	16	measurement	849	16	18.51	864
01/F0027	R8.10.2	S	1	measurement	531	1	12.36	532
01/F0054	J2.15.0	S	1	measurement	750	1	13.39	751
01/F0068	R7.4.2	S	16	warmup	—	—	—	—
02/F0019	J2.10.24	S	16	warmup	—	—	—	—
02/F0036	R7.4.2	S	16	warmup	—	—	—	—
02/F0045	R7.4.2	S	1	measurement	184	1	7.35	185
02/F0071	R8.10.2	S	1	warmup	—	—	—	—

Trial	System	Mode	P	Stage	Conf.	Unknown	Join, s	Stored
03/F0046	J2.10.24	P	1	measurement	11208	1	7.62	11209
03/F0048	R7.4.2	S	1	measurement	299	1	8.15	300
03/F0063	R8.10.2	S	1	measurement	830	1	15.19	831

Complete paired contrasts, trial memberships and unavailable-pair counts are in `followup/derived/statistics.json`. Individual event, telemetry and syscall observations remain in the campaign directories and trial table. The main article presents synchronization diagnostics.

References

- [1] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi:10.1093/biomet/26.4.404.
- [2] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction: CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. URL: <https://lean-lang.org/papers/lean4.pdf>, doi:10.1007/978-3-030-79876-5_37.
- [3] Redis contributors. Redis 7.4.2 append-only file implementation: `src/aof.c`, 2025. Source code, release 7.4.2. Accessed 2026-09-23. URL: <https://github.com/redis/redis/blob/a0a6f23d997b024689ba157916837f493a593a34/src/aof.c>.
- [4] Redis Go client contributors. go-redis v9.7.0 command-specific socket timeout calculation. Pinned source code. Accessed 2026-09-24. URL: <https://github.com/redis/go-redis/blob/ed37c33a9037483ad2a6b1042e5eb6df89009a1c/redis.go#L470-L478>.
- [5] The Lean Developers. Lean 4.24.0 release notes, October 2025. Released 2025-10-14. Source tag: `leanprover/lean4 v4.24.0`. Accessed 2026-09-23. URL: <https://lean-lang.org/doc/reference/latest/releases/v4.24.0/>.
- [6] The NATS Authors. nats.go v1.39.1 JetStream fetch and acknowledgment implementation. Source code. Accessed 2026-09-23. URL: <https://github.com/nats-io/nats.go/blob/4ba1afe8709b048ae89888f0f94da7eb8497e7ec/js.go>.
- [7] The NATS Authors. NATS Server v2.10.24 file store: `server/filestore.go`, 2024. Source code, release v2.10.24. Accessed 2026-09-23. URL: <https://github.com/nats-io/nats-server/blob/1d6f7eaf0bd966dd02c95b8d5f624868f1a62544/server/filestore.go>.
- [8] The NATS Authors. NATS Server v2.15.0 file store, 2026. Pinned source code. Accessed 2026-09-24. URL: <https://github.com/nats-io/nats-server/blob/eb763679aa3c24a40dcd3012aa046ad1996d851c/server/filestore.go>.